

職務経歴書

氏名：

H. E

生年月： 1978 年 09 年（満 47 歳）

住所： 東京都八王子市 在住

最寄駅： JR 中央線 西八王子駅

最終学歴： 金沢工業大学 工学部 電子工学科 卒業

概要：

- ・ iOS のアプリ開発に携わるようになって 14 年くらいになり、iOS 絡みであれば大体のことはできます。
- ・ 独りで企画デザイン～リリース／運用／保守の傍、受託・SES 経由で大手からベンチャー系まで、アプリ開発を請負う仕事をさせて頂いてます。
- ・ iOS 以外でも MacOS アプリの開発を行うこともあります。
- ・ 最近では Android アプリの開発に携わったり Web アプリケーションの開発、WebAPI 設計・構築、スクレイピング等、Web に関する開発に携わる機会も多いです。
- ・ プロジェクトリーダーとして仕事させて頂いた時期もありますが Scrum 等の仕事経験して以降、技術屋に徹しています。
- ・ Tech リード／コードレビューも行えます。
- ・ ロール（職務・権限）には沿って仕事を進めます
- ・ 既存コードがあれば、それに沿って開発を進められます。
- ・ 「仕方がない」、「とりあえず」といった場当たりの仕事はやりません。

職歴：

2015/04 - 現在

スキルについて：

★×3つ：調べながら実装可能 ★×4つ以上： 大体できる
★×2つ：若干怪しいと自覚している

 Apple (約 14 年) ★★★★★

ゲーム以外の大抵のことはできる。
iOS/iPadOS 以外（MacOS 等）もアプリ作成可

 ★★★★★

 ★★★★★

 ObjC ★★★★★

関連して経験して内容：

- ・ UIKit、Cocoa、SwiftUI、AVFoundation（動画撮影、動画再生、録音、録音再生、スロー再生、スロー録画、動画比較）、ジオフェンスコー

ディング、位置情報、写真の合成（オブジェクトをイメージの上に配置して合成して1枚の写真にする、また配置するオブジェクト情報の管理と、その再現）、課金（Apple 課金システム、PayPay、カード決済）、通信処理（Get、Post、Delete、Put、Patch）、リアルタイム通信、ソケット通信、データ管理（CoreData、または Realm、SQLite）、App Intents、Notification Service Extension、Apple Watch 対応、Mac アプリの作成、Map 機能（Map上にカスタマイズしたピンを立てる等）、Apple 認証、他 SNS 認証、Swift Concurrency、Combine、Darkモード対応、等

Android (約 3 年) ★★★★★

一人でアプリを作ってリリースできるくらいにはスキルあり。Kotlin、Java 両方対応可

Web App ★★★★★

求められれば構築は可能



Java (約 5 年) ★★★★★

業務外でもかなり勉強して認定資格まで取得ただし、知識は Version 5.0 で止まっている。

Microsoft 系 (約 5 年) ★★★★★

VB で業務アプリ（海外工場向け）、C# で社内向けの Web アプリを構築、等

DataBase 系 (約 10 年) ★★★★★

EBS を含め、OracleDB (7.0 ~ 10g まで) との関わりが長い。

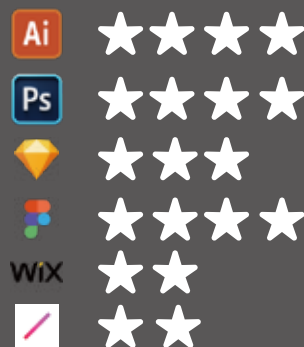
RubyOnRails で MySQL をよく使う。



UI/UX 系



デザイナーと仕事する機会が多く、彼らができることは自分でもできるようにトレーニング中。現在、デザインに関するレジュメを作成中



その他の技術

その他、変わり種で触ったことある技術を記載

- Bitrise / 自動テスト・自動ビルド
- OS2/WARP
- 16Bit 版、Visual Basic (Version 4.0)
- 16Bit 版、Microsoft Access (Version 2.0)
- 16Bit 版、Microsoft Excel マクロ
- Tcl/TK (Unix 用の GUI)
- PLSQL+Apatch
- C 言語／汎用機 (JSL コマンド)

業務経験

携わったことのある業務について記載。

- | | |
|----------|----------|
| 工場系： | 金融 |
| • 生産管理 | • 証券業務 |
| • 在庫生産 | その他： |
| • BOM 管理 | • 特許システム |
| 会計： | • グリーン調達 |
| • 調達管理 | • 不動産系 |

システム開発／ QA 業務：

- 要件定義から詳細設計まで対応可能
- 単体テスト～システムテスト設計も対応可能

Ruby On Rails を使ったシステム構築

- 営業リストの作成。複数のビジネスサイドや Google Business から情報を取得して、業種別にカテゴライズ。メール／電話番号／住所をリスト化
- ゴルフ場の予約枠の金額比較サイトの作成。競合ゴルフ場との枠の金額を比較できるもの
- 自社内での Flutter アプリ開発

検索ワード：

使用したことあるツールなどを列挙。

AIによるマッチング機能による、ファイル検索された時にアンマッチなることを避けるために使用したことがある技術やツール、キーワードを思いっただけ追記。

Bitrise, Firebase, Reveal, Charles, JsonExpoter, BugsSnap, BitBucket, Github, Vue.js, PHP, Ruby, AWS, AWS SDK, LINE アプリ開発, HTML5, CSS, Docker, Oracle, CoreData, Cocoa, BootStrap, WebRTC, CocoaLumberjack, Laravel, GCP, Amazon EC, Amazon ECS, AmazonEC3, Ruby On Rails, Spring, iBeacon, IIS, Oracle, OracleEBS, npm, nuxt.js, Material Design, .Net, Bitrise, Circle CI, Git-Flow, CVS, Subversion, IntelliJ, Access, Android, Swagger, Backlog, RedMine, JetBrains, Atlassian, Jira, Confluence, Trello, Bitbucket, Slack, Microsoft Teams, Scrum, Miro, Illustrator, Photoshop, Sketch, MySQL, PostgreSQL, SQLServer, TypeScript, Swift, SwiftUI, Cocoa, Java, Seasar, Seasar2, Geofence, ChatGPT, Microsoft, Word, Excel, Access, スプレッドシート, Google Workspace, Google Sites, Google Forms, spreadsheet, Postman, Amazon Cognito, Cocoa2D, VBA, Visual Basic, SQL, OCA, CVS, Mercurial, SourceTree, MacOS, OS/2 Warp, DB2, RADIUS, DBMS, PL/SQL, Realm, Figma, Webpack, AFNetwork, Alamofire, Nokogiri, OneSignal, SwiftJSON, Socket, VPN, VNC, Logic Pro, サイボウズ, Zeplin, Youtube, yaml, yarn, WordPress, Wix, WinSCP, Wireshark, SVG, Sonar, Skype, Sharepoint, Shake, RestAPI, RestfulAPI, Google Analytics, LeanPlum, Notion, Node.js, MVC, MS Office, MinGW, Microsoft Project, JP1, Homebrew, Gradle, Google Tag Manager, Google Play Console, Google Docs, Flutter, IBM, Amazon RDS, Amazon SQS, Agora, Tencent, AR, VR, R2.framework, Struct2, VB.net, 釣り銭機, iPadOS, Vision Pro, Proxy Man, リクルート, Heroku, iOS, iPhone SDK, Core Data, Core Bluetooth, Core Audio, XML, JSON, JSONDecode, Swift, SwiftUI, Java, Oracle, Tcl/Tk, AIX, 生産管理, グリーン調達, 社内システム, 調達, イントラマート, Objective-C, ObjC, OpenTP1, JavaScript, シェル, Bsh, Csh, SJCP, SJC-P, SJC-WC, Metrix, Alphablox, Twitter, Facebook SDK, Adobe, Instagram, Nokogiri, Kotlin, Firebase Realtime Database, Redis, Elasticsearch, TensorFlow, PyTorch, GitLab, Vagrant, Xcode, Eclipse, MariaDB, Netlify, Bootstrap, jQuery, Docker Compose, Jenkins, Travis CI, Azure DevOps, Terraform, Ansible, Rust, Scala, Haskell, Meteor, Django, Flask, Svelte, Cassandra, CouchDB, Firestore, Azure, DigitalOcean, Asana, Monday.com, Basecamp, ClickUp, Anaconda, Keras, D3.js, Tailwind CSS, Sass, Puppet, Chef, Consul, UIKit, Combine, AVFoundation, MapKit, ARKit, SpriteKit, SceneKit, Metal, SnapKit, RxSwift, Moya, Kingfisher, Charts, Lottie, CocoaPods, MVC (Model-View-Controller), MVVM (Model-View-ViewModel), MVP (Model-View-Presenter), VIPER, Singleton, Observer, Delegate, Factory, Adapter, Strategy, Command, Sketch, OWASP, Zscaler, Amazon Waf, Terraform

モバイルアプリのゼロトラスト通信の PoC サポート	
2025年5月～2026年05月(約12ヶ月)	職位／役割：PMO
業務内容： モバイルアプリのゼロトラスト通信（ZSDK／ZPA Browser Access）および認証基盤（Auth0 + AWS Lambda Authorizer）に関する技術検証と、関連する iOS 端末側セキュリティ（RASP 堅牢化）の調査。また、作成する検証アプリを Figma にてデザイン。	
プロジェクト概要 クラウドベースのゼロトラスト（Zscale をモバイル・Web アプリ通信で利用できるかの検証を目的とした社内 PoC に参画。iOS/Android/ Web アプリ 3 プラットフォームを統一アーキテクチャで揃え、Auth0（OIDC / OAuth2）+ JWT+ AWS サーバーレスバックエンドでの実機検証を主導。 プロジェクト内では AI 支援開発を積極活用。Claude Code / ChatGPT / GitHub Copilot / Cursor を工程別に使い分け、設計書・実装・検証スクリプト一貫して AI 併走で作成。	
使用技術・ツール	
・ Figma	
・ 言語 / フレームワーク : Swift, SwiftUI, TypeScript / Next.js / React, Terraform	
・ バックエンド : AWS Lambda（Python / Node.js）, API Gateway, DynamoDB, ECS Fargate, ALB	
・ AI 活用 : Claude Code, ChatGPT, GitHub Copilot, Cursor	

～ 2025 年

3D UI 搭載カスタマー向け iOS アプリ開発実績	
2025年7月～2025年02月(約6ヶ月)	職位／役割：リーダー
業務内容： ※機密性の高いプロジェクトのため、詳細は記載できません。	
プロジェクト概要 3D UI を採用したカスタマー向け iOS ネイティブアプリの開発に従事しました。Unity 製の 3D UI コンテンツをアプリに組み込む手法（Unity as a Library: UAAL）を採用し、Unity エンジニアと協力してリッチなユーザー体験を実現しました。アプリ側では Swift/SwiftUI で画面を構築し、バックエンドの Web API と通信して取得したデータを Unity コンテンツへ反映する仕組みを実装しました。	
役割・担当業務	
・ ネイティブアプリ側開発：iOS アプリ部分の開発全般を担当し、Web API 経由のデータ通信機能や、取得データを Unity 側に渡す処理を実装しました。	
・ Unity 連携基盤の構築：Unity 公式のライブラリ埋め込み機能である UAAL を使い、ネイティブ（Swift）と Unity 間の双方向通信を構築しました。Unity.framework（Objective-C++ 形式）をプロジェクトに統合し、ネイティブ側がコントローラとして Unity コンテンツの状態管理やアクション実行を行えるようにしました。	
・ SwiftUI 対応：Unity コンテンツの表示には通常 UIViewController を用いますが、本案件では SwiftUI を採用。SwiftUI 画面上に Unity の UIViewController ラッパーを組み込む形で、最新の UI フレームワークでも Unity コンテンツを表示・操作できるようにしました。	
技術上の課題と対応	
・ Apple シリコン非対応による開発効率低下：当初使用していた Unity 2019.3 系（Unity 3 系）は Arm64（Apple シリコン）に非対応でした。そのため M1/M2 Mac 上の iOS シミュレータで Unity コンテンツを動作確認できず、実機ビルドに時間が掛かる状況でした。また Unity.framework のサイズが大きく、アプリのビルド時間が数分～十数分と長時間化する問題がありました。	
・ ビルド時間短縮の施策：開発効率改善のため、Unity コンテンツを XCFramework 化し、パッケージ管理を **Swift Package Manager (SPM)** に移行しました。	
・ これにより Unity 部分を都度ビルドし直す必要がなくなり、ビルド時間を大幅に短縮することに成功しました。	
・ Unity 新バージョンへの対応：開発途中の 11 月頃に Unity コンテンツをメジャーアップデート（Unity 2020 LTS 以降のバージョン）し、Apple シリコンに正式対応しました。これにより iOS シミュレータ上でも Unity コンテンツが実行可能となり、実機に依存しない効率的な開発・デバッグが行えるようになりました。	
開発上の工夫・成果	
・ JSON プロトコルによる柔軟な通信設計：UAAL 経由の通信は文字列のみのやり取りとなるため、ネイティブ⇄Unity 間で JSON 形式のコマンド文字列を定義しました。Unity 上のオブジェクト情報や実行アクション、状態を表すメッセージフォーマットを設計し、ネイティブ側で Unity 内オブジェクトの生成・操作・状態管理を実現。要求や仕様変更にも柔軟に対応できる抽象化されたインタフェースとなりました。	
・ CI 導入の検討：継続的インテグレーション（CI）環境で Unity コンテンツのビルド管理を自動化することも検討し、一部パイプラインを構築しました。コンテンツサイズの制約から完全な SPM 管理・自動ビルドには至りませんでした。	
使用技術・ツール	
・ 言語 / フレームワーク : Swift, SwiftUI, Unity (UAAL)	
・ ライブラリ管理 : Swift Package Manager (SPM), XCFramework	
・ プラットフォーム : iOS (iPhone/iPad), Apple Silicon Mac (開発環境)	
・ その他 : Objective-C++ (Unity 統合部分), JSON プロトコル設計, Web API 通信	

野球のバッティング／ピッチング解析アプリ（iOS） 2023年10月 ～ 2024年4月(7ヶ月) 職位／役割：プログラマ 業務内容： iOS アプリの新規開発（β版アプリからのリプレース）。このアプリは、野球のバッティング／ピッチングの動画を AI で解析するもので、以下の業務を担当しました。 アプリ全体のアーキテクチャ設計 Apple Store Kit 2.0 を使用した内部課金機能 動画解析機能、動画撮影、動画再生以外のアプリ機能全般 WebAPI の設計とモックアップ作成 技術的アプローチ： 簡単な設計を要望され、ReactiveKit を使用した MVVM と通信系モジュールの DI（Concurrency）対応を実施 画面遷移はストーリーボードと R.swift、セグエを用いて実装を簡素化 備考： 知人エンジニアの紹介で参画。参画当初（2023 年 10 月）は、開発進捗 5%、デザイン未完成で、2023 年 11 月末リリース予定でした。既存の iOS エンジニア 4 名が稼働していたが、生産性が低く、アーキテクトも未整備。PM も機能しておらず、開発体制の立て直しを実施。既存の 3 名をプロジェクトから外し、知人の iOS エンジニアに参画を依頼。開発範囲の見直しやプロダクトデザインの矛盾点の洗い出しを行い、WebAPI の再設計とモックサーバを構築。2023 年 12 月に動画解析開始以外のアプリ開発を終了。 しかし、ステージングサーバにデプロイされた WebAPI に再設計内容が反映されておらず、ステージングサーバの改修とアプリとの疎通確認に 2 ヶ月を要した。リプレースした動画解析機能の品質がβ版よりも悪いことが発覚し、担当者が離脱。解析コードを見直し、3 フレームのずれを発見して修正。その他の雑務も多く担当し、2024 年 4 月にリリース。4 月末にクライアントから資金不足を理由に 5 月是非稼働、6 月からの再参画を依頼されたが、連絡なく契約終了。11 月～3 月までの平均睡眠時間は 3 時間。	投資を学ぶための配信型学習アプリ (iOS) 2023年1月 ～ 2023年8月(9ヶ月) 職位／役割：プログラマ 業務内容： iOS／Android アプリの新規開発。iOS のプログラマとして参加。それぞれ 2 人体制。 投資で実績ある投資家が、投資を学びたい人向けに授業を行う内容で、配信する投資家 1 人に対して同時に 1 人～数万人まで同時配信を行うためのプラットフォーム（Agora）をアプリに組み込みました。「配信者」と「視聴者」の両方が使用する機能を 1 つのアプリで実現しました。 アプリ全体のアーキテクトの設計と Apple Store Kit を使った内部課金機能、主要機能（ホーム画面、動画撮影、動画再生）以外のアプリ機能のすべてを担当。また、WebAPI の設計とモックアップの作成も担当しました。SwiftUI と Combine（Concurrency）を使用してアプリを構築しました。 備考： サーバサイドを担当しているエンジニアからアプリ側で API の設計を依頼され、Swagger を使用して全 32 個の API を作成し、モックサーバに接続してアプリを構築しました。また、Bitrise によるデプロイの自動化も実施しました。 参画当初、Android アプリエンジニアが 1 名のみだったため、知人のエンジニアをプロジェクトに紹介。また、途中で iOS のエンジニアが抜けたため、再度知人のエンジニアを紹介しました。4 ヶ月ほどで開発を終了しましたが、配信テストや WebAPI の単体テスト不足が原因で結合テストで大量の不具合が発生しました。配信確認が取れた段階で、講師となる「配信者」からアプリでの配信が分かりづらいとの意見があり、PC 上で動作する Web アプリに開発をシフトしました。Web アプリ開発にはフルスタックエンジニアを紹介し、プロジェクトは 3 ヶ月後にローンチされました。 開発環境： ・プログラム言語 : Swift5 / UIKit（iOS15 - 16） ・使用ツール : Xcode、Swagger、パケット系ツール ・ライブラリ : CocoaPods、R.Swift、Alamofire、Kingfisher、AgoraRtcEngine_iOS ・その他 : SwiftLint を導入していましたが、Lint の指摘を無視してプッシュされることが続いたため、SwiftFormat と Pre-Commit を導入し、ランチにプッシュする前に強制的にコードを修正できるように対応
--	---

スマホプリントアプリ開発補助（iOS） 2019年11月 ～ 2022年12月(37ヶ月) 職位／役割：プログラマ 業務内容： スマホの写真や素材を使用した年賀状印刷サービス用の iOS アプリ開発のエンハンスに参加しました。このアプリでは、ユーザーが年賀状の宛名を入力し、4,000 点以上のテンプレートからデザインを選択、スマホに保存していた写真を挿入して年賀状を作成できます。作成した年賀状は、印刷所に WebAPI 経由で転送され、ユーザーの自宅に郵送されるか、そのまま投函されます。元々のプロダクトは Web アプリケーションで作成されていたものをネイティブアプリ化するという珍しいプロジェクトでした。 プログラマとして、新規機能の追加やプロダクトのリファクタリングに従事しました。プロダクトは 2014 年にローンチされたもので、Objective-C と Swift のコードが混在していたため、これをリファクタリングし、Firebase のダイナミックリンクにも対応しました。 備考： 2019 年の年末リニューアルリリースを計画していたが進捗が遅れていたため、期間限定で参画。2019 年のリリース後はリファクタリングと新規エンハンスに従事。2022 年 11 月に他年賀状プロダクトとの統合が決まり、正社員エンジニアの増員を受けて契約終了となりました。 大量のテンプレートを一括でダウンロードする機能をリファクタリングしました。その他、通信モジュールを Moya から Alamofire に変更（Moya のサポートが終了していたため）、PromiseKit を使用していた箇所を Async/Await に移行、AWS SDK を使用して AWS S3 へのアクセスでタイムアウトが発生していた箇所を修正、Objective-C で組まれていたホーム画面を Swift に変更し、各パーツをコンポーネント化、大量のテンプレート一覧表示画面、iPhone 内の写真表示画面で UITableView と UICollectionView の描画を高速化し、これらのコンポーネント化も実施 開発環境： プログラム言語 : Swift5 / UIKit（iOS13 - 15） 使用ツール : Xcode、パケット系ツール ライブラリ : CocoaPods、R.Swift、Alamofire、Moya、SnapKit、Bond、ReactiveKit、ZIPFoundation、PromiseKit、AWS SDK

アスリート向け就業支援用の LINE アプリの作成	
2022年10月 ～ 2023年2月 (5ヶ月)	職位／役割： プログラマ
業務内容：	
LINE 向けにアスリートの就業支援を目的とした Web アプリケーションを作成しました。アプリの内容は、LINE ID をベースにアスリート自身が競技情報を Firebase に保存、呼び出し、更新できるものです。保存および更新された情報はスプレッドシートに出力され、運営元（当時は某自治体を想定）が確認し、要件ごとにオファーされることを想定していました。	
備考：	
新しくアプリケーションを作成するため、最新リリースされたばかりの Nuxt 3.0 と Firebase 9.0 を使用しました。情報が少なかったため、公式マニュアルを見ながら作業を進めました。また、LINE アプリから Web アプリが起動できるように設定しました。	
開発環境：	
動作環境：Nuxt.js 3.0 / Firebase v9 / Vuetify 3.0 / TypeScript 他	
開発ツール：Visual Studio Code、Gitpod	
その他：スプレッドシート Script	

モバイルオーダーアプリの開発支援	
2022年9月 ～ 2022年12月 (3ヶ月)	職位／役割： プログラマ
業務内容：	
飲食店向けの Saas 型のモバイルオーダーシステムの開発支援に従事。プロダクトの概要としては、お店側は iPad アプリを利用してメニュー管理／レジ機能／オーダー機能／売り上げ管理等、お店の運営をトータル的にサポートします。飲食店に訪れたお客様は LINE アプリ経由で注文が入り、厨房の注文伝票にサーマルプリンターに出力されて、調理が始まり、提供。会計時にはレジにの iPad アプリにて会計します。	
先方のビジネスプランとして App Store 上で販促したい意向で iOS アプリ化を希望されており、iOS アプリエンジニアとして参画。開発では WKWebView 上で該当する Web アプリを動作させられるように対応。また、Web アプリからサーマルプリンタを接続するにあたり IP アドレスの設定が必要なため、スタート精密製のサーマルプリンターとの接続をアプリ側で処理できるようにスタート製 SDK を組み込み。それにより、アプリをダウンロードして内部データを作成することですぐに使用できるように対応。また、Web アプリからスタート精密がプリンタ出力用の JSON をアプリが JS 経由で受け取った後に iOS SDK で出力できるようにラップ機能を提供。アプリの申請リリースも担当。	
プログラム言語：Swift5 / UIKit (iOS14)	
使用したツール：Xcode、Safari 開発ツール	
ライブラリ関係：CocoaPod を使用し、R.Swift、Alamofire、SnapKit、OneSignal	

相席ラウンジ型・居酒屋店舗レジシステム開発補助 (iOS)	
2019年8月 ～ 2019年10月	職位／役割： プログラマ
レジシステム（Web アプリケーション）とグローリー製釣銭機（iOS と釣銭機を中継するスター精密製のブリッジ装置）を接続する iOS アプリを作成しました。iOS アプリでは、アプリ内に Web サーバを構築し、Web アプリからアプリに HTTP リクエスト経由で釣銭機を制御できる機能を提供しました。将来的に店舗が増えることを考慮し、App Store にも出稿（現在はダウンロード不可）しました。	
プログラム言語：Swift5 / UIKit (iOS12)	
使用ツール：Xcode、Safari 開発ツール	
ライブラリ関係：CocoaPods を使用し、Alamofire、SnapKit、m-bridge framework（提供は Framework 形式でしたが、内部で GitHub にコードを登録し CocoaPods からインストールできるように修正）を登録して CocoaPods からインストールできるように修正）	
テスト機： 釣り銭機、m-Bridge、RS-232 ケーブル	

中古車販売店向け・中古車メディアデータ分析ツール作成	
2022年6月 ～ 2022年11月	職位／役割： プロダクトオーナー
業務内容：	
Ruby on Rails を使用して Web アプリケーションを作成しました。このアプリは、中古車販売サイトに車情報を掲載している中古車販売業者向けのもので、クライアントにとって競合となる業者の販売車情報を照合し、提供することを目的としています。毎日、クライアントの在庫情報とクライアントが指定した競合の在庫情報をクローリングし、販売情報として一覧にまとめ、それを元に販売戦略に活用していただきます。	
環境：	
プログラム言語：Ruby on Rails	
開発ツール：Cloud9、Docker	
サーバ：Heroku	
データベース：AWS	

パーソナルジムやフィットネス専用の顧客管理・予約管理アプリ (iOS / Android)	
2022年4月 ～ 2022年12月 (8ヶ月)	職位／役割： プログラマ
業務内容：	
パーソナルジムやフィットネス向けの顧客管理・予約管理できる Saas 型サービスのスマホアプリ化を担当。元々、Web アプリとして提供されているサービスであるが、スマホアプリ化することによってサービスのブランディング化をハイありたいと相談され、iOS / Android でアプリ化開発では WKWebView 上で該当する Web アプリを動作させられるように対応。カスタマ向けにプッシュ機能（OneSignal）を実装し、Web アプリでログインした際に OneSignal で管理する UserId を Web アプリに JS 経由で保存できるように対応。Universal 対応を含め実装。また、Android 版も作成またクライアントの希望により、希望するクライアントには独自アプリを提供して、クラアントのブランディグサービスを提供したいとのこと。iOS の場合、プロジェクトの中でジムごとにビルドターゲットを分けてパイナリを出力できるように対応。アプリの申請リリースも担当。	
プログラム言語：Swift5 / UIKit (iOS14)	
使用したツール：Xcode、Safari 開発ツール	
ライブラリ関係：CocoaPod を使用し、R.Swift、Alamofire、SnapKit、OneSignal	

建物管理の業務オペレーション用 iOS アプリの開発	
2019年6月 ～ 2019年7月 (2ヶ月)	職位／役割： プログラマ
業務内容：	
点検・検針・清掃報告などを入力するアプリを構築しました。アプリに保存した情報はクラウドサーバにアップロードされ、その後、自動で報告書が作成されます。これにより、報告の品質向上とビルメンテナンス業務の時間短縮が可能です。プロダクトには既に Android アプリが存在しており、それをトレースしながら iOS アプリを作成しました。ビルメンテナンスの性質上、電波が入らない施設も多く、オフラインでも動作し、デバイスがオンラインに移行した際にアップロードを再開できるように対応。取り扱う情報は、動画、音声ファイル、写真、テキストデータなどです。また、通信した情報をキャッシュする仕組みも必要でした。	
備考：	
アプリのアーキテクチャを構築し、そのアーキテクチャを基に、OS エンジニア 2 名で 2 ヶ月間で開発を完了。	
開発環境：	
プログラム言語：Swift5 / UIKit (iOS12)	

使用ツール：Xcode、Safari 開発ツール
ライブラリ関係：CocoaPods を使用し、Alamofire、SnapKit、Realm、SwiftDate、Kingfisher、ObjectMapper、ReachabilitySwift

2016 年～2018

不動産チャット接客アプリ（iOS）の開発補助（iOS）
2017年7月～2019年6月(2ヶ月) 職位／役割： テックリード／プログラマ

業務内容：
不動産屋に来店せずに部屋を探し、不動産の営業担当とチャットで調整しながら内見の申し込みまで行えるアプリを担当しました。チャット欄では、ユーザーが登録時に入力した条件に基づき、AI が該当しそうな物件を日々提供する機能や、チャットボットによるアンケート機能、引っ越し予定日が近づけば通知して物件探しの再開を促す機能を提供しました。さらに、リッチリモートプッシュやサイレントプッシュ機能を使ってユーザビリティの向上を図りました。リアルタイムチャットには Pusher、効果測定には Leanplum を採用しました。

既存アプリの大規模改善のために参画し、当初 1 ヶ月の見積もりだった改修がコードの状態の悪さから 3 ヶ月かかりました。1 ヶ月かけてリファクタリングを行い、デザイナーと相談しながら各 UI のコンポーネント化を進めることで、参画当初の機能改善を 1 週間でリリースできるように改善しました。アーキテクチャはクリーンアーキテクチャを意識したプロトコル指向を採用し、通信系処理のエラー時メッセージ出力を含め DI 化することでエンハンス効率を最大化しました。

備考：
当時、LINE を意識したチャット機能を目指しており、スタンプ機能以外は遜色ないものを提供できていたと考えます。正社員の 3 年目と 10 年目のエンジニアに iOS アプリ開発を指導し、プロトコル指向が身についたことで、一人でアプリのエンハンス開発ができるようになったタイミングでプロジェクトを離脱しました。また、他の不動産系会社から同社のチャットシステムを使ったアプリのリリース相談を受けた際には、Swift のプロトコルを用いて既存システムからデザインとアイコン、WebAPI の接続先を差し替え、3 日でアプリの新規登録申請を行った実績もあります。

開発環境：
プログラム言語：Swift3、4、4.2 / UIKit（iOS9～iOS12）
使用ツール：Xcode
ライブラリ関係：CocoaPods を使用し、R.Swift、Alamofire、SnapKit、ObjectMapper、Pusher、TTTAttributedLabel、Kingfisher、ObjectMapperAdditions、Adjust、Facebook SDK、JTAppleCalendar、Realm

海外レストラン予約アプリの App Intent（Siri Kit）を使用した予約機能の追加

2016年11月～2016年12月(2ヶ月) 職位／役割： プログラマ
業務内容：
海外の食べログに相当する、アメリカのレストラン予約アプリの拡張を担当しました。拡張内容として、インストール済みのアプリに登録されたレストランを Apple Maps 上で選択した際、レストラン情報をアプリから呼び出し、予約情報を表示してそのまま Maps 上で予約できる機能を追加しました。当時、日本では App Intent を実装したアプリは存在せず、また Apple 社とビジネス契約が必要であったため、おそらく日本の開発者の中では初めての試みだったと思われます。iOS エンジニア 2 名、サーバサイドエンジニア 1 名で対応しました。

開発環境：
プログラム言語：Swift4.2 / UIKit（iOS10）
使用ツール：Xcode

地図アプリのエンハンス（iOS）
2019年4月～2019年06月 職位／役割： プログラマ

業務内容：
某インターネットサービス会社が提供する地図アプリのエンハンスに参加。スクラム形式でエンハンスを進めるプロジェクトで内部でペアプログラミングを始めて経験しました。集まっているメンバーの経験に大きな差があったため、うまく業務が回らず苦労しました。

備考：
他プロジェクトを抱えていたため実装は主だってペアを組んでいた方をお願いしてましたが、年齢差なのか経験値の差なのか、うまく彼を導くことができず反省が残るプロジェクトでした。

プログラム言語：Swift5 / UIKit（iOS12）
使用したツール：Xcode

海外ホテル予約アプリの開発保守（iOS）
2017年7月～2018年3月(7ヶ月) 職位／役割： プログラマ

業務内容：
某大手ホールディングス会社が買収した海外ホテル予約サービスのアプリ開発をサポートするために参画しました。その中で、フィリピンのホテル予約サービスを担当しました。業務内容は、現地のエンジニアが開発したブルリクエストを確認することでした。
しかし、アプリのリプレース計画は予定通りに進まず、半年遅れの状況でした。急遽、Thomas という名前で開発に参加し、2 ヶ月間で 960 時間稼働しました。

備考：
ホテル予約サービスが他の海外資本に買収され、買収が決定した段階でプロジェクトが終了
実際には、アプリのリプレースができるかどうかが買収の鍵でした。
"Thomas" として頑張ったので、ボーナスが欲しかったです。

開発環境：
プログラム言語：Swift3、4、4.2 / UIKit（iOS9～iOS12）
使用ツール：Xcode
ライブラリ関係：CocoaPods を使用し、R.Swift、Alamofire、ObjectMapper、Kingfisher、ObjectMapperAdditions、SwiftLint、Facebook SDK、SwiftlyJson、GoogleMaps、KeychainAccess、SwiftCop、ActiveLabel、WARangeSlider

携帯小説用アプリ（iOS）のリプレース
2016年5月～2016年7月(3ヶ月) 職位／役割： テックリード／プログラマ

業務内容：
出版社が運営する「携帯小説」というジャンルのコンテンツを扱う iOS 版アプリのデザイン刷新に参画しました。携帯小説とは 2004 年頃にガラケーから始まったサービスで、誰でも携帯電話から小説を投稿し、他のユーザーが閲覧できるサービスです。ユーザーの 9 割は 10 代～30 代の女性で、PV が多い小説は書籍化されるスキームもありました。スマホアプリは 2012 年にリリースされ、広告収入だけで月 400 万～500 万の売上があります。
私の担当は、Objective-C で実装されている通信系モジュールを Swift 2.2 で再実装することでした。通信系のレスポンスモデルも JSON Export ツールを使用してすべて Swift の ObjectMapper クラスに書き換えました。また、SnapKit を使った画面作成、新デザインに合わせた基底クラスの作成、小説をジャンル別にタブで切り替える大規模画面の作成、動画広告の導入も担当しました。リードテックとして 3 名のチームを率い、実装方法を指南しました。

開発環境：
プログラム言語：Swift 2.2、使用ツール：Xcode、ライブラリ関係：CocoaPods を使用し、Alamofire、ObjectMapper、SDWebImage、SnapKit、SliderMenu 等

2010 年～2013 年

広告代理店で展開するサービスで作成されたアプリの開発、メンテナンスを担当（iOS）

2013年10月～2016年3月(29ヶ月) 職位／役割： プログラマ

業務内容：
*** レジ：
レジアプリチームに配属。システムのポイント型アプリを開発
某レジアプリで会計した時に付与されるポイントを管理し、溜まったポイント
を使用できるアプリの開発を担当。iBeacon によるジオフェンス機能を実装。

ゴルフ場予約アプリ：
ゴルフ場を検索して予約するアプリの開発とメンテナンス。Objective-C で実
装されていたものに対して、エンハンスでは Swift2 系を使って実装。この頃か
ら、Masanory による Autolayout による描画を行うようになる。

医療系アプリ（Swift）のプロトタイプ開発：
男性の妊活アプリのプロトタイプを開発。採取した精子をレンズで拡大し、動
画撮影して動画解析用サーバに転送。解析結果を閲覧できるアプリ。2017 年に
ローンチし、様々な賞を受賞して 2020 年に終了。動画解析サーバについては
他会社が担当しており、HTTP 通信に知己がなかったためこちらで対応。

プログラム言語：Objective-C（iOS3.1～）
開発ツール：Xcode、Subversion、Github、Cocoapods
使用ライブラリ：AFNetwork、Mantle、Masanory、SDWebImage 等

大手広告代理店のアプリ開発基盤の立ち上げ

2011年6月～2012年3月(7ヶ月) 職位／役割： 開発基盤整備

業務内容：
大手広告代理店で、外注していたアプリ開発の内製化に伴い、開発基盤立ち上
げに参加しました。外注で開発していた各チームが代理店内のフロアに集め
られ、横断的にコミュニケーションを開始し、共有ドキュメントやコードの共
通化を担当しました。特に、当時まだ主流でなかったリモートプッシュ機能に
ついて調査し、送信サーバの作成、送信手順の作成までを準備しました。また、
アプリリリースを補助しました。

プログラム言語：Objective-C（iOS3.1～）、PHP
開発ツール：Xcode、Subversion

感想：
アプリ開発チームには大きく 3 つの派閥が存在しており、それぞれ非常に独特
な文化を持っていました。そのうちの一つは議論の際に大声で怒鳴る文化があ
り、若いエンジニアが萎縮することも多々ありました。チーム間の話し合いに
は間に入ったり、同じ目線で仕事をしてもらうために仕事以外でも飲み会を計
画したりと、蟻りを解消するための努力をしました。最終的にはチームメンバー
の入れ替えを提案しましたが、気に入らないコードがコミットされていると、
それを削除し徹夜で再実装するリードエンジニアの習慣を辞めさせることはで
きず、何人ものエンジニアが去るのを見送るのが非常に辛かったです。力のあ
るエンジニアが特別扱いされる現場でしたので、トラブルも多かったです。

動画配信アプリの開発

2012年9月～2013年6月(9ヶ月) 職位／役割： プログラマ

業務内容：
当時 Youtuber という言葉すらない時期に、YouTube、ブログサイト、
Facebook などから動画をクローリングし、スポーツやエンタメ、健康などに
ジャンル分けして動画を楽しむというコンセプトで開発を開始しました。当時
のスマホの通信は 3G 回線のみであり、動画を満足に再生できない状況だった
ため、動画のキャッシュが求められました。サーバ側から動画の詳細ページの
URL は届くものの、動画そのものの URL は入っておらず、iOS 内で対応しリリー
スしました。さらに、YouTube の動画を保存できるようにし、YouTube API
を使用してプレイリストを作成できる音楽アプリも開発しました。

備考：
電車や地下鉄での動画再生は厳しかったようです。
2013 年に所属していた部署が解体されることが決定し、それに伴いアプリも
クローズしました。

感想：
当時、Youtuber という言葉がなく、ヒカキンが YouTube 第三世代（自分のキャ
ラクターを売っていく方式）と呼ばれていた時期でした。Megwin というチャ
ンネルを運営している方に担当が企画を持って会いに行きましたが、他の企画
の誘いが多く断られてしまいました。振り返ってみると、その方は先見の明が
あり、5 年くらい先が見えていたと思います。うまく企画を形にはできなかった
ものの、貴重な経験だったと感じます。

個人で iOS アプリケーションの作成

2010年3月～2011年5月

内容：
5 本のアプリを Apple Store に公開し、合計で約 100 万円の売上を達成しまし
た。

備考：
アプリの開発と公開にあたって、私は 20 冊以上の関連書籍を購入しました。
しかし、多くの著書やブログメディアは表面的な内容しか記載されておらず、
理解を深めることが難しい状況でした。そんな中、Erica Sadun の著書「The
iOS Developer's Cookbook (Developer's Library)」に出会い、非常に感銘を受
けました。この本は詳細な解説と実践的なアドバイスが豊富で、私のスキル向
上に大いに役立ちました。以来、Github で Erica Sadun 氏をフォローし、彼
女の最新の知見を取り入れるよう努めています。
また、SEO や口コミの難しさに直面しました。有料版アプリとして販売して
いましたが、恐らく競合だと思いますがよくない書き込みをされることが多く、
それにより全く売れなくなることが多々ありました。この頃はまだ広告がない
状態でして、売り上げを上げようと思ったら有料販売するしかなかった時代で
した。対策として、スクリプトで Google や Yahoo と言ったフリーメールアド
レスを用いて 500 個くらい AppleID を作ってレビューを上乗せしたり、アプ
リ内でタップされるとアプリ名で Google 検索 WebAPI を実行させる等、がん
ばりました。しかしながら、米 Apple 社から直接電話で口頭注意を受けて過剰
な対応を控える様にしました。

大手電機メーカー社内システム保守

2009年4月～2010年01月(8ヶ月)	職位／役割：SE
業務内容：	
大手電機メーカー内で使用する社内システムの運用・保守を担当。チームをまとめつつ複数のシステムを管理しました。システムのほとんどは Web アプリ化されており、Java（一部 C#）や Java サブレットで稼働しています。各システムの保守案件の管理、スケジューリング、要員のサポート、メンバーの育成に従事しました。	
担当していた Web システム：	
・社内福利厚生申請システム（Java・イントラマート）	
・社内 IP 電話管理システム（Java・Seaser2・Sip サーバ）	
・社内安否確認システム（Java、Struts）	
・調達管理システム（イントラマート・Page Base）	
・グループ会社管理システム（Spring、PLSQL）	
C#：	
・社員検索システム（C#）	
その他：	
・社内 Windows サーバへのリリース作業	
メンバー、最大 10 名	

管理するシステムも多く、バグの修正依頼も頻繁に発生。全てエクセルにて管理しており、正確な事象／原因／作り込んだ理由／類似する不具合／仮対応／本対応で区分けしてバグを管理しました。この手法により、不具合の発生が激減しました。バグを作り込んだエンジニアについてはリポジトリを遡って個別に調査し、私個人の秘匿情報として管理しました。

東京大学用特許申請システム

2007年12月～2008年02月(3ヶ月)	職位／役割：SE
大手電気メーカーが自社の特許システムを基に廉価版システムを作成し、東大向けに提供されていたシステムの改修を担当しました。以下の作業を実施しました。	
・基本設計書の修正	
・詳細設計書の作成	
・単体テスト仕様書の作成／単体テストの実施	
・結合テスト仕様書の作成／結合テストの実施	
プログラム言語：Java1.2（Struts1.1）	
インフラ：Linux／Oracle8i／XML	

大手コンビニ大規模業務システムの再構築

2005年10月～2007年6月(20ヶ月)	職位／役割：SE
業務内容：	
プロジェクトの概要は、大手コンビニが 1 店舗～各エリア（市区町村）ごとの情報を、営業／調達／売上／品ごとの販売個数など、ありとあらゆるデータを取り込み、それを多次元で解析し、営業や経営戦略を行うためのデータを作成するシステム構築です。分析用のデータは各コンビニ店舗からデイリーで送られてきたものを取り込み、スーパーコンピュータ並みの処理速度を保つサーバ 3 台を使って一晩で全てのデータを集計、各軸での対比データを作成します。これを Web アプリでアクセスする形で提供します。	
プログラム言語：Java 1.4（Alphablox IBM 製のフレームワーク）	
インフラ：UNIX／Essbase（多次元解析を可能とするミドルウェア）／Oracle 10g	
開発規模：全体 150 名、所属チーム 10 名	

備考：

プロジェクトの途中からリーダーを兼任し、営業向けの画面機能を担当しました。プロジェクト開始以来、一度もリリースされていなかったが、自分が担当に入ったことで一番早くリリースが実現しました。

現代（2024 年）においても、他業種や大手の会社であっても、月の売上やデイリーの製品販売数、そのエリア／地域の天気情報に至るまで、情報をまとめレポートにするまで 1 日以内で作成できるような会社は少ないと思います。

ちなみに食事でお弁当を食べる際には、競合店舗のものを机に広げるとプロパーの方がいらっちゃって、「そのお弁当、どこのお弁当ですか？どこで買われたんでしょう？」とプレッシャーをかけられ、該当するコンビニでしか買い物できない状況でした。プロジェクト離れてからは半年は、その系列のお店で買い物はで

グリーン調達に基づくシステムの開発・保守。調達部品の環境規制物質の調査・管理のためのシステム管理保守

2008年3月～2009年03月(12ヶ月)	職位／役割：SE
大手電気メーカー内での使用を想定し、サプライヤごとに調達対象の部品や製品に含まれる人体や自然環境に悪影響を与える物質の有無を調査するシステムを開発。サプライヤに調査依頼をメールで行い、その結果をメールで受信し、システム DB に格納。システム側で指定する不含有率をクリアしているかどうかを確認し、クリアしている場合はその証明書を発行。該当サプライヤの部品が原因で自社製製品が納品できなかった場合、裁判に提出する証明書として使用する。	
対象となる環境物質の規約には RoHS や WEEE、REACH 規制等が含まれる。	

担当システム：

社内福利厚生申請システム（Java・イントラマート）

社内 IP 電話管理システム（Java・Seaser2・Sip サーバ）

社内安否確認システム（Java、Struts）

調達管理システム（イントラマート・Page Base）

グループ会社管理システム（Spring、PLSQL）

社員検索システム（C#）

インフラ：

プログラム言語：Java1.4、Struts1.1、Excel VBA

インフラ：Windows2000 サーバ / Apache（SSL） / SQL Server2000

備考：

当初は予算がなく、私一人で管理していたが、システムに予算がついた段階で他部署に引き取られていった。

グリーン調達についてはこちらをご覧ください。

https://www.env.go.jp/policy/env-disc/com/com_pr-rep/rep-ref06.pdf

CAD ファイル管理理システム

2007年7月～2007年11月(5ヶ月)	職位／役割：SE
業務内容：	
大手電気メーカーが代理販売する CAD パッケージ「Metrix」のシステム運用・保守を担当しました。このシステムは独自のデータ更新用コマンド「MQL」を持っており、MQL を使用してデータの更新を行ったり、専用のツールを作成したりしました。担当業務には、システムの運用管理、保守作業、新しい機能の追加、ユーザーサポートなどが含まれていました。	
プログラム言語：Java1.3、Tckl/TK	
インフラ：Windows2000 サーバ / Apache（SSL） / Oracle9i	

カード利用顧客情報管理システムの改修移行作業 2004年12月～2005年09月(10ヶ月)
業務内容： カード会社の顧客情報管理システムの移行プロジェクト。エンドユーザーは割賦販売や消費者ローン等のクレジット事業を営む企業を会員とする信用情報機関であり、システム移行中に開発メンバーの情報を基に与信を問い合わせを行うこともしばしば。改修内容は F 社製の COBOL で実装されていたシステムを H 社製の COBOL と H 社製のミドルウェアに移行する計画であった。しかし、F 社製と H 社製のコンパイラには大きな違いがあり、そのため移行に失敗した。さらに、当初のインターネット対応と同時進行で進める必要があったため、難易度が高いプロジェクトとなった。
規模： 全体 300 名 所属 QA チーム 10 名

ネットワーク中継サーバのミドルウェア作成 2004年05月～2004年8月(4ヶ月)	職位／役割： プログラマ
業務内容： 金融機関間で電信を中継する処理を C 言語で開発。ミドルウェアに OpenTP1 を使用。	
開発環境： ・ Windows2000 ・ VB6.0	
備考： ・ 銀行合併に伴い、前現場から当時勤めていた会社に出戻り	

暗号化・復号化ファイルの送受信システム開発 2002年6月～2002年8月(2ヶ月)	職位／役割： プログラマ
業務内容： 通信の暗号化／復号化のコンポーネントを VB6.0 にて作成	
開発環境： ・ Windows2000 ・ VB6.0	
備考： ・ 銀行合併に伴い、前現場から当時勤めていた会社に出戻り	

証券代行システム開発 2001年6月～2002年5月(11ヶ月)	職位／役割： プログラマ
業務内容： 証券代行システム部内の業務アプリの新規開発、保守開発 1．証券を発行するための業務アプリケーションの作成 2．オペレータが使用する業務システム（Access がほとんど）の保守開発 3．顧客データや、入力ミスしたデータの修正	
開発環境： ・ OS2/WARP ／ WindowsNT ／ 汎用機 ・ 16bit 版 VB4.0、Access 2.0、16bit 版 Excel ・ JCL	
備考： ・ 当時、所属していた会社から銀行系システム会社へ出向しての業務 ・ 16Bit 版のエクセルの場合、1 シートに 1 万 6 千行まででありながら、マクロで 1 万 6 千枚の帳票を出力する処理を作ったことがしんどかった。 ・ 16Bit 版 VB で兆の桁の計算処理を作ったことがしんどかった。 ・ 16Bit 版 AccessVBA で 100 万行のコードをテスト項目を作って全てテスト。3 ヶ月くらい全く人と喋らなかったことあり。 ・ 人生において、最も辛い体験の 1 つ	

ダイレクトインターネットバンキングの検査業務 2004年9月～2004年11月(3ヶ月)
業務内容： インターネットバンキング向けに作られたシステムの検査業務に従事しました。元々は QA なしで進んでいたプロジェクトでしたが、バグが多発し対応が追いつかないため補助として参加しました。具体的には、開発チームが作成した検査設計書やテスト項目の検査、テスト項目の修正、テストの消化を実施しました。 備考： 作成されているテスト項目の内容が不明確で、10 人が見れば 10 通りの解釈ができるような曖昧なものでした。この問題を検査資料に指摘すると、開発チームの評判を落とすことになるとして怒られました。この経験を通じて、仕事観やエンジニアとしてのあり方について深く考えさせられました。

工場内の生産・在庫管理システムの運用保守 2002年12月～2004年3月(15ヶ月)
業務内容： とある工場のシステム管理を行っている会社に出向。様々な製品や部品が製造される工場で、製造現場ごとに異なるシステムが存在し、日々、生産情報がシステム間で連携されて生産管理と在庫管理が行われていた（現在は廃工）。工場で扱われる情報はすべて Oracle 社の製品（Oracle 7、8、9i、Oracle Application、EBS 等）で管理されており、一部はクライアントアプリでアクセスするケースもあったが、大部分は Oracle Apache + PL/SQL で、HTTP から PL/SQL オブジェクトにアクセスし、記載されている HTML コードで画面を描画していた。基本的に SQL で作業することが多く、SQL はかなり打ち込んだ。 現場からの要望に基づき、システムやデータの修正を行った。月末の棚卸し時には、現場内の部品の数とシステムで管理される在庫データに差が発生する場合、部品 ID ごとにトランザクションを発生させて調整を行う作業にも従事。工場内のデータには東南アジアや中国の工場からもアクセスがあり、当時の回線で HTTP 通信を行うと大幅な遅延が発生するため、Windows アプリを提供し、ODBC 接続でデータを更新していた。 開発環境： プログラム言語：PL/SQL（HTML、JavaScript）、Tcl/Tk データベース：Oracle 7.3、Oracle 8i、Oracle 9i、OCA OS：Unix（B シェル、C シェル） 使用ツール： ObjectBrowser

仕事への取り組みについて：

私は個人として「やり切ること」をモットーに、強い気持ちで仕事に取り組んでいます。プロダクトの成功のためであれば、必要に応じて生活を削って作業を進めることもあります。ただし、周りに同じ働き方を強いることはありません。あくまで自己判断での行動です。また、「決めること」もモットーとして持っており、特に私が個人でプロダクトを設計するときの指針になっています。

エンジニアとしての力を発揮することも大事ですが、チーム／組織として成果を最大限に上げるために、どうしたらよいか、私自身がどう貢献できるかを常に念頭に置きながら仕事に取り組んでいます。若く、これから力を発揮していくエンジニアのサポートができればと考えています。そのために、自分の中で「気を付ける」「心がける」ことがいくつかあります。

「職域を超えない」ように気をつけています。ただし、プロジェクトの進め方に問題があって内部で解決できない場合はイニシアチブを取ることもありますし、個別に仕事を引き受けることも多いです。（ただし、ハラスメントには全力で抵抗します）

「美味しい仕事（楽しい仕事、やりがいある仕事）」は取らない、独り占めしない。新しい技術や考え方はプロジェクトにとって有効であればチャレンジすべきだと考えています。楽しくやりがいのある仕事は若いエンジニアに、雑多な仕事は自分が引き受けるように心がけています。ただし、誰もやりたがらなかった場合、喜んで担当させて頂きます。

「円滑にプロジェクトが回るように成果物を確認」することを心がけています。アウトプットを確認し、曖昧な箇所は早めにチェックし、起案する際には状況を見て会議等でそれとなく伝えるようにしています。

「エンジニア間のマウントの取り合いはしない」ように気をつけています。マウントを取らなければプロジェクトが進まない場合や、私のエンジニアのアイデンティティを否定される場合はやむを得ませんが、基本的には避けるようにしています。

Apple 系の開発エンジニアとして：

Apple 社が提供するデバイス向けのアプリケーション開発に携わるようになって、14 年くらいになります。きっかけは 2010 年 1 月、参画していたプロジェクトを離れ、1 ヶ月の休養を挟みながら新しい案件を探していた時に起きたリーマンショックでした。市場から案件が急速に消え、エンジニアのレイオフが大量に発生し、Java エンジニアの単価が当時のレートから半額にまで下がった状況を見た時、新しいことを始めるべきだと考えました。色々調べた結果、iPhone のアプリストアでアプリケーションを販売して収益を上げている個人開発者がいることを知り、翌日には真っ白な MacBook と iPad を購入したのを覚えています。

当時の iPhone SDK / Objective-C は、Java 5.0 系と比較して約 10 年遅れている印象を受けました。ガーベージコレクションはなく、コード内でリリースを明示的に行う必要がありました。それはまだ良かったのですが、コレクション関数が存在せず、CoreData もなく、UserDefaults でひたすら頑張って実装した覚えがあります。

仕事を辞めてから半年、試行錯誤の末にアプリを 2 本リリースしてみたものの、何冊も本や資料を読み漁っても iPhone SDK の中身が理解できず、非効率的なプログラムの書き方になっていることがわかり、学習に限界を感じました。Java の世界に戻ろうと転職活動を再開したタイミングで、Erica Sadun 氏の著書に出会い、目から鱗が落ちるような思いで理解が一気に進みました。同時に、一人で悩むより

もその分野のエンジニアに会いたくなりました。アプリ開発の要員を募集している会社やエージェントを探し始め、2011 年 3 月 11 日の震災から 3 ヶ月後、東京駅近くの案件にアサインされました。Erica Sadun 氏に出会っていなければ、今日の私はなかったと思います。

アーキテクトの設計者／テックリードとして：

iOS 開発には MVVM、MVP、Clean Architecture、TCA などのキーワードがあります。これらは初期は PHP から、その後に JavaScript 系から持ち込まれた考え方で、Web 系エンジニアが理解しやすくするために採用された印象が強いです。気持ちはわかります。元々 Java エンジニアである私にとっても、MVC 以外の構成には違和感があり、さらに V と C が同じクラス (UIViewController) で扱われることにも違和感がありました。解決する仕組みは iOS SDK 内に準備されていますが、コードの可読性が悪くなる傾向があります。しかし、上記のアーキテクチャが至高かと言われればそうではありません。これらのアーキテクチャは整理されていますが、実装するコードの量が増え、その再利用性は低く、似たコードが増えていくだけに思えます。ちなみにこれらのアーキテクチャに関して構成の理解はあります。その上で無駄が多い印象を持っています。ただし、SwiftUI に宣言的 UI が導入されたことにより、MVVM を採用する機会も増えました。ViewModel に継承やジェネリクス等、共通性を求めると意図としない動きをすることもあり、そこは MVVM に習っています。また SwiftUI を使用する際はプレビューをしっかりと記載すること、できればユースケースごとにテストコードを記載することも意識するようになりました。

個人的には Swift 3 以降、プロトコル指向による徹底した抽象化と、Storyboard (1 つの ViewController に 1 つの Storyboard) や Xib を活用して記載するコードを極力少なくすることを信条としています。コードが少ないとバグも減ります。プロトコルを使うことでプロジェクト内のルールをエンジニアに示す効果もあります。実装中にコード補完ができないと辛いので、R.swift を導入し、カラー、文字列、イメージ、ファイル名を管理します。また、テーブルビューやコレクションビューで扱うセルに関しては抽象化し、「追加」が発生する際には他からプログラムをコピー＆ペーストして必要箇所だけ変更することで動作するプロジェクトを目指します。画面間の遷移については、Storyboard を使用する場合、Segue や UnwindSegue による遷移を推奨します。Apple 系の開発を長くやってきて感じることは、View から開発を始めることを想定されていることです。Storyboard 上に Segue を実装することにより Storyboard の内容を見るだけでどこの画面に遷移するのかを判断できます。そこにコードリーディングは必要ありません。チームで開発を行う際には、コードレビューを実施することがほとんどだと思います。コーディング規約を設けつつ LINT を導入してルールに従って実装してもらうようにしておりますが、年々、スケジュールが厳しくなっておりますので、メンバーには Git にプッシュする前に自動で Swift-Format で設定したルールに従ってコード整形するように設定をお願いしています。

また、オブジェクト指向の観点から「final class」の記述は推奨しません。なぜこういう書き方をするのか不思議に思い、調査したところ final をつけることによってオーバーライドしたクラスがないと動的ディスパッチ (継承関係をチェック) を避けるようになるので、パフォーマンスが向上するというものでした。オブジェクト指向の観点から見ると愚行でしかなく、「final 修飾子をつける」ということ自体、そのクラスの設計しそを考慮するとほぼさわれないクラスであると認識します。そんなにパフォーマンスが必要なら、Static 領域での処理を検討したり Struct を使うことを検討すべきであり、final をつけることに関しては懐疑的です。

コード x AI について

PoC を通じて、AI を活用した開発検証を行いました。Figma で作成したデザインをもとに、アプリ内容の約 8 割を AI で実装することを試みました。デザイン自体を AI で生成することも可能ですが、今回は PoC でありながら外部展示を予定していたため、ブランディングを含めて私の方でデザインしました。ブランド要素をデザインへ反映しつつ、人の感性に訴えかけるようなデザインを AI が構築することは、現時点では難しいと考えます。実装面では、内部ロジックや共通コンポーネントを含む、SwiftUI を使った MVVM+UseCase アーキテクチャを私の方で設計しました。そのうえで、設計ルールを AI にインプットし、ルールを守らせながら実装させることを試みましたが、当初はうまくいきませんでした。AI が生成したコードを解析した結果、当時最新であった Swift 5.9 系や Swift 6.0 系のコード、Observable を用いた実装に関する学習データが少なかったことが原因だと考えられます。また、Swift では一般的ではないアーキテクチャを採用していたことに加え、AI の Swift における Protocol の理解が不十分だったことも要因だと考察します。

そこで方針を変更し、アーキテクチャの基底部分は私の方で実装し、AI エージェントにはそれをコピーまたは拡張する形で実装できるようにしました。また、MVVM+UseCase で守るべきルールや実装方法をプロンプトで定義しました。しかし、AI は学習が不十分な箇所では別の手段でコードを生成しようとするため、まずプロンプトを読み込ませたうえで簡単な ToDo アプリを構築させました。その結果をレビューし、問題のある実装をアンチパターンとしてプロンプトに反映し、最適化を行いました。同様の作業を 4～5 回繰り返すことで、画面遷移、メッセージ表示、アラート表示、通信処理、内部データ処理、UseCase 以外の部分については、最低限 AI に実装を任せられる状態にできました。そのため、最初の 1 画面だけは私の方で作成し、それを参考にしながら AI エージェントに実装を任せました。

AI エージェントには、View の構造や ViewModel の構造といったテンプレートを遵守させながら、15 画面ほどを作成させました。途中、UseCase の中身についても指示を与え、実装を行わせました。最終的なデバッグと評価は私の方で行いましたが、バグ修正については AI に指示を出し、改修させました。途中で構造を読み違えることもあったため、その都度アンチパターンを追加しました。AI には UseCase まわりの修正を担当させ、私は View の描画崩れを修正する形で役割分担したところ、通常であればテスト実施と不具合修正に 2 週間ほどかかる作業を、3 日で完了させることができました。

全体を通して最も時間がかかったのはデザインで、約 2 週間でした。一方、開発自体はプロンプト調整を含めて約 1 週間、デバッグとテストは 3 日で完了しました。アプリ内には、多少コンポーネントを多重に組み込む箇所はありましたが、無理なりアクティブ処理を排除して単純化したこと、Figma で UI と UX を明確に AI へ提示したこと、実装範囲を明確にしたことが、開発効率化につながったと分析しています。つまり、設計に関わる箇所を明確に定義できたことが、成功要因だったと考えます。なお、構築した iOS アプリをベースに、Android アプリへの移行も AI を活用して実施しました。その結果、アプリ全体の約 6 割をほぼ AI だけで移行できました。要因としては、iOS 側のアーキテクチャが Android で推奨される MVVM 構造と近かったことが挙げられます。

新しいモデルが追加され、試す機会も増えていますが、品質の低いコードが生成されるケースが増えたと感じています。おそらく、バイブコーディングの影響で学習可能なコード量が増えた一方で、コード品質にばらつきが出ているためだと考えます。これは大きな課題であり、今後は独自に学習データを選定し、目的に合ったモデルを構築する必要があると考えています。